Part 3

- Notes (Refer Last Section):

Docker - Container to Container Communication

Key Concepts:

- **Isolation:** Each *Docker* container acts as an isolated environment with its own file system, processes, and networking stack. Much like separate apartments in a building, one container cannot see or access another container's resources unless a specific network connection is established.
- **The 'Localhost' Misconception:** When you are outside a container, `localhost` refers to your host machine. However, **inside a container**, `localhost` only refers to that specific container itself. This is why calling `localhost:5000` from a frontend container to reach a backend service will fail—the backend process does not exist inside the frontend's isolated environment.
- **Bridge Networking:** To allow communication, you must use a network. While a default *bridge network* exists, it does not easily allow containers to communicate by name. Creating a **user-defined network** enables **service discovery**, where containers can talk to each other using their container names rather than fragile, changing IP addresses.

Why Container-to-Container Communication is Needed:
Modern applications are not a single container but a combination of multiple services.
For example:

- Backend container → handles API logic
- Database container → stores data
- Frontend container → interacts with backend
  These services need to talk to each other (e.g., backend querying the database). Since containers are isolated, they **cannot communicate by default**, so Docker networking is required to make the application work as a whole.

Advantages of Container Networking:

1. **Service Discovery:** IP addresses often change during restarts, but container names remain constant, ensuring reliable communication.
2. **Security:** By isolating services within a network, you can prevent unauthorized external access (e.g., keeping a database hidden from the public while allowing the backend to connect to it).
3. **Port Collision Prevention:** Containers run in isolated environments, so multiple services can use the same internal ports without conflicting.
4. **Scalability:** You can easily add more instances (e.g., multiple backend containers) behind a single service name, facilitating better load management.

This foundation is critical for implementing **NGINX Reverse Proxy** and, eventually, **Kubernetes Services**, where these networking concepts are applied in a more automated and scalable production environment.

NGINX

NGINX is a high-performance **web server** and **reverse proxy server** designed to handle large-scale traffic efficiently. It follows an **event-driven, asynchronous architecture**, which allows it to manage thousands or even millions of concurrent connections with very low resource usage.

What does NGINX actually do?
At a high level, NGINX sits between the **client (user/browser)** and your **application servers (backend services)**. Instead of the client directly calling your backend, the request first goes to NGINX, which then decides where to send it.

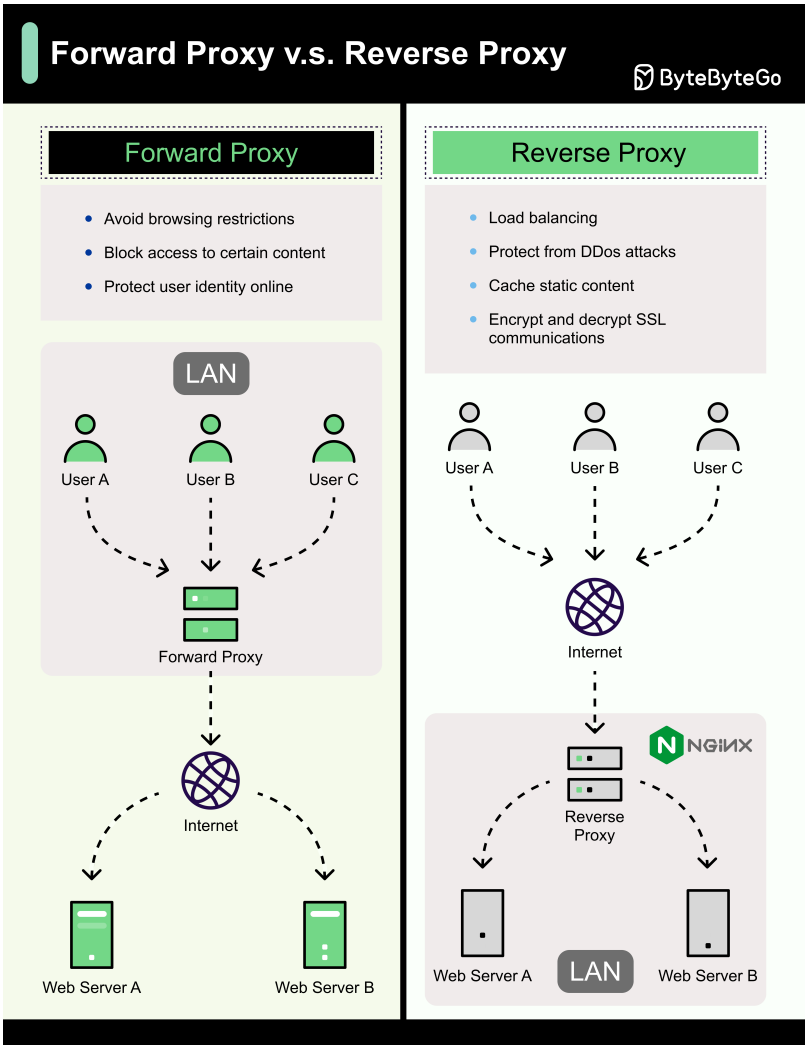
Reverse Proxy (Core Concept)

- NGINX mainly acts as a **reverse proxy**, meaning it works on behalf of the **server**.
- **Flow:** `Client → NGINX → Backend Server → NGINX → Client`
- **What it handles:**
  - Accepts incoming requests from users
  - Routes them to the correct service (backend, frontend, etc.)
  - Returns the response back to the client
- **Why is Reverse Proxy useful?**

- **Abstraction of backend services**: Clients don't know how many servers exist or where they are. NGINX hides this complexity.
- **Smart Routing**: Example:
  - `/` → frontend
  - `/api` → backend
  - `/images` → static server
- **Load Balancing**: If you have multiple backend servers, NGINX distributes traffic among them.
- **Security**: Internal services are not directly exposed to the internet.
- **Performance Optimization**: Can handle caching, compression, and serve static files efficiently.

Forward Proxy (for comparison)

- A **forward proxy** works on behalf of the **client**, not the server.
- **Flow:** `Client → Forward Proxy → Internet`
- **Use cases:** VPNs, Corporate networks, Content filtering / blocking
- Here, the destination server sees the proxy instead of the actual client.



Setup Overview (MERN + NGINX + Docker Compose):
We are running a **MERN stack application** using Docker, where:

- **MongoDB** → database
- **Backend** → API server
- **Frontend** → UI (React/Vite)
- **NGINX** → reverse proxy (entry point)
  All services are connected using a **common Docker network (** `mern-net` **)**.
  We start everything using: `docker compose up --build`.
- **nginx.conf (Reverse Proxy Configuration)**
  - This file defines how incoming traffic is routed.
  - **Purpose:**
    - Serve frontend at `/`
    - Route API requests (`/api`) to backend
    - Act as a **single entry point**
  - **Configuration:** */nginx/nginx.conf*: (as used in video)
    - ```
      events {}

      http {
        # Allow larger request bodies (e.g., file uploads)
        client_max_body_size 10M;

        server {
          listen 80;

          # Route frontend requests
          location / {
            proxy_pass http://frontend:5173;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-Proto $scheme;
            proxy_buffering off;
          }

          # Route API requests to backend
          location /api {
            proxy_pass http://backend:5000;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-Proto $scheme;
      ```
      ```

```
        proxy_connect_timeout 5s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
      }

      # Health check endpoint
      location = /nginx-health {
        return 200 "OK";
        add_header Content-Type text/plain;
      }
    }
  }
```

- **docker-compose.yml (Service Orchestration):** Defines all services and how they connect.
  -
```
services:
  mongo:
    image: mongo:6.0
    container_name: mongo
    restart: unless-stopped
    volumes:
      - mongo-data:/data/db
    networks:
      - mern-net

  backend:
    build:
      context: ./server
      dockerfile: Dockerfile
    container_name: backend
    restart: unless-stopped
    env_file:
      - ./server/.env
    depends_on:
      - mongo
    networks:
      - mern-net

  frontend:
    build:
      context: ./client
      dockerfile: Dockerfile
      args:
```

```
        VITE_API_URL: "/api"
    container_name: frontend
    restart: unless-stopped
    depends_on:
      - backend
    networks:
      - mern-net

  nginx:
    image: nginx:alpine
    container_name: nginx
    restart: unless-stopped
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    ports:
      - "80:80"
    depends_on:
      - frontend
      - backend
    networks:
      - mern-net

volumes:
  mongo-data:

networks:
  mern-net:
    driver: bridge
```

How Everything Works

- **Step 1: Startup**
  - When we run: `docker compose up --build`
  - Docker:
    - Builds frontend & backend images
    - Pulls Mongo + NGINX images
    - Creates a network: `mern-net`
    - Starts all containers
- **Step 2: Networking Magic**
  - All containers are inside the same network: `mern-net (bridge network)`
  - So they can talk using **container names**:

    - `backend` → reachable as `http://backend:5000`
    - `frontend` → `http://frontend:5173`
    - `mongo` → `mongodb://mongo:27017`
- **Step 3: User Request Flow**
  - When user opens browser: `http://localhost`
- **Step 4: Request hits NGINX**
  - Because:

    ```
    ports:
      - "80:80" #HOST_PORT : CONTAINER_PORT
    ```

  - NGINX is exposed to host
- **Step 5: NGINX decides routing**
  - **Case 1: Frontend request**
    - `/`, NGINX routes to: `http://frontend:5173`
  - **Case 2: API request**
    - `/api/users`, NGINX routes to: `http://backend:5000`
- **Step 6: Backend → Mongo**
  - Backend internally calls: `mongodb://mongo:27017`. This works because of Docker network.
- **Full Flow:**

```
            User (Browser)
                 ↓
            localhost:80
                 ↓
            NGINX (reverse proxy)
                 ↓
  ┌──────────────┬──────────────┐
  │              │              │
Frontend      Backend        (API)
                 ↓
               MongoDB
```

Kubernetes Declarative

The transition from **imperative** to **declarative** configuration in Kubernetes marks a fundamental shift in how applications are managed in production.

1. Imperative vs. Declarative Approaches:

- **Imperative (Basic/Part 1):** Involves running manual commands like `kubectl create deployment` or `kubectl create pod`. This approach is time-consuming, prone to human error, and difficult to track in complex environments.
- **Declarative (Standard):** Involves defining the desired state of the system in **YAML files**. You tell Kubernetes *what* you want the state to be, and the system works to reconcile the current state with your desired configuration.

2. Key Advantages of Declarative Configuration:

- **Version Control:** YAML files can be stored in code repositories (like Git), allowing teams to track changes, audit who made them, and revert to previous versions.
- **Drift Detection & Self-Healing:** Kubernetes controllers continuously monitor the cluster. If a Pod is killed or a node fails, the controller detects the "drift" from the desired state and automatically reconciles it by spinning up new replicas.
- **Environment Consistency:** The same manifest can be promoted across different environments (dev, stage, prod) simply by applying the file, ensuring consistency.
- **Scalability:** It is significantly easier to manage large clusters with hundreds of components using structured YAML files than by executing individual commands.

3. Core Kubernetes Objects Recapped:

- **Pod (The Building Block):** The **Pod** is the smallest deployable unit in *Kubernetes*. It is essentially an isolated environment for your containers. Importantly, pods are **ephemeral** (short-lived), meaning their IP addresses change frequently, which makes them unreliable for direct, stable communication.
- **ReplicaSet (The Reliability Layer):** A **ReplicaSet** acts as a controller that ensures a specified number of identical pod replicas are running at all times. If a pod crashes or is terminated, the *ReplicaSet* automatically detects this drift and recreates the pod to maintain the desired state.
- **Deployment (The Management Layer):** A **Deployment** is a higher-level controller that handles *ReplicaSets*. It provides declarative updates, such as

rolling updates and **rollbacks**, allowing you to manage different versions of your application infrastructure without manual intervention.

- **Service (The Network Endpoint):** Because pods and *ReplicaSets* are dynamic and their IPs keep changing, the **Service** object is used to provide a **stable network endpoint**. It acts as a consistent gateway—consisting of a *DNS* name and a *ClusterIP*—that forwards traffic to the underlying, healthy pods, effectively abstracting the volatility of the pod layer away from the user.

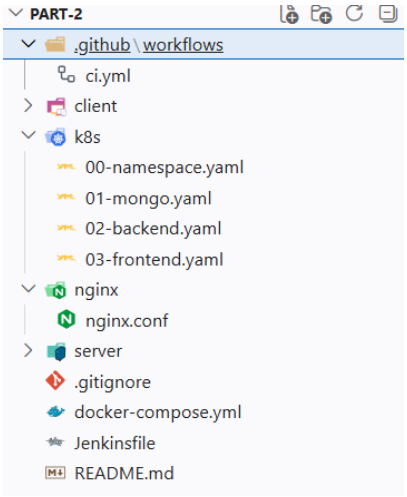
Namespaces: These are **virtual clusters** or organizational "folders" within a single physical Kubernetes environment. They are essential for isolating resources, allowing teams to separate environments like development, testing, and production effectively.

Labels and Selectors: These serve as the mechanism for resource discovery. **Labels** are metadata (key-value pairs) attached to objects (like Pods), while **Selectors** allow Services to identify and group these objects to route traffic correctly.

- **Minikube Driver Theory:** The instructor briefly explains the importance of the *driver* (specifically `--driver=docker`) when starting *Minikube*. This is theoretically significant because it defines the container runtime environment, ensuring the *Kubernetes* cluster can interact with the same local environment where your container images are built.
- **The Controller Reconciliation Loop:** While mentioned as part of the move to declarative, the theory of the *Reconciliation Loop* is fundamental. It explains *why* the declarative approach works: the *Kubernetes* controller constantly monitors the current state of the cluster and performs actions to align it with the *desired state* defined in your YAML files.
- **Service Type Theory:** In this segment, the instructor touches upon *Service* objects and their role in networking. The theory is that *Pods* are ephemeral (short-lived), meaning their IP addresses change frequently. *Services* provide a stable, consistent network endpoint (via *ClusterIP* in this context) that allows different components of an application (like the frontend and backend) to find each other regardless of which *Pod* instance is currently handling the request.
- **Image Management via Local Registry:** The video discusses why you need to build or tag your images locally (e.g., `backend:local`) for *Minikube* to recognize them. Theoretically, this highlights how *Kubernetes* nodes pull images—if they cannot reach an external registry (like *Docker Hub*), they must

have access to a local source that matches the image specification in your manifest.

Folder Structure:



commands ran
6.5: docker build --build-arg VITE_API_URL=http://localhost:5000/api -t frontend:local ./client
minikube image load backend:local
minikube image load frontend:local
minikube image ls

- create namespace, then run mongo, then backend and frontend
  kubectl apply -f k8s/00-namespace.yaml
  kubectl apply -f k8s/01-mongo.yaml
  // to check if mongo is running
  kubectl -n devops-part-3 wait --for=condition=ready pod -l app=mongo --timeout=180s
  kubectl -n devops-part-3 get pods -l app=mongo -o wide

kubectl apply -f k8s/01-backend.yaml
// similarly u can check if backend is running or not using above commands
// to check svc
kubectl -n devops-part-3 get svc backend -o wide

kubectl apply -f k8s/01-frontend.yaml

// check all pods and services in that namespace
kubectl -n devops-part-3 get pods,svc -o wide

//now we have to do port forwarding
kubectl -n devops-part-3 port-forward svc/backend 5000:5000

kubectl -n devops-part-3 port-forward svc/frontend 5173:5173

Configuration and Secrets in Kubernetes

It is essential to avoid hardcoding sensitive or environment-specific data into our application code. This allows for easier updates without needing to rebuild container images. The instructor distinguishes between two types of information:

1. ConfigMaps: Used for **non-sensitive** information, such as API URLs or feature flags. These are stored as key-value pairs and can be consumed by pods as environment variables or mounted as files within a volume. The speaker notes that these are not encrypted and are visible to anyone with cluster access.

2. Secrets: Designed for **sensitive** data, including passwords, tokens, and database URIs. Secrets are stored as Base64 encoded strings in YAML manifests. While not a true encryption method, the speaker highlights that this is a best practice compared to plain text and emphasizes that secret YAML files should **never be pushed to public repositories**.
The practical implementation involves:

- **Creating the Config and Secret objects:** Defining them as standalone Kubernetes resources.
- **Injecting into Deployments:** Modifying the backend deployment to reference these resources, allowing the application pods to pull configuration dynamically at runtime.
- **Verification:** Using `kubectl get configmap` and `kubectl get secret` to confirm the objects exist, and verifying the application functionality within the browser to ensure the backend is connecting correctly via these non-hardcoded values.

Ingress

Ingress acts as a **smart router at the edge of a Kubernetes cluster**. It provides a **single entry point** to expose multiple services and route traffic based on defined rules.

Why Do We Need Ingress?
Limitations of Basic Service Types

- **ClusterIP**

    - Internal communication only
    - Not accessible from outside the cluster
    - Pods are dynamic → IPs keep changing
- **NodePort**
    - Exposes services on ports (30000–32767)
    - Hard to manage for multiple services
    - Not scalable or production-friendly
- **Port Forwarding**
    - Only for testing/debugging
    - Not suitable for real-world usage

Benefits of Ingress

- **Single Entry Point**
    - No need to expose multiple ports for each service
- **Path-Based Routing**
    - Example:
        - `/` → Frontend
        - `/api` → Backend
- **Host-Based Routing**
    - Example:
        - `app.example.com`
        - `api.example.com`
- **SSL Termination (HTTPS)**
    - Handles HTTPS at the Ingress level
- **Load Balancing**
    - Distributes traffic across multiple pods
- **Scalability**
    - Easily supports multiple services and microservices architecture

Traffic Flow:
`User Browser -> Ingress Controller -> Ingress Resource (rules match) -> Kubernetes Service -> Pods`
Key Idea:

- Ingress **does NOT directly talk to pods**
- It routes traffic via **Services**

Core Components

1. **Ingress Resource (YAML):** Defines routing rules
   - Key fields:
     - `ingressClassName` → specifies controller (e.g., nginx)
     - `rules`
       - `host`
       - `http.paths`
2. **Ingress Controller:** Actual component that:
   - Reads Ingress rules
   - Routes traffic accordingly
   - Example: NGINX Ingress Controller

Practical Implementation (Minikube)

- **Step 1: Enable Ingress Controller:** `minikube addons enable ingress`
- **Step 2: Verify Controller:** `kubectl get all -n ingress-nginx`
- **Step 3: Create Ingress YAML:** Example (`04-ingress.yml`):

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-app-ingress
spec:
  ingressClassName: nginx
  rules:
    - host: man-app-demo.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
                  number: 80
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: backend-service
```

```
                    port:
                      number: 5000
```

- **Step 4: Start Tunnel (Important for Local Setup):** `minikube tunnel`
  - Creates a bridge between your local machine and cluster
  - Required for exposing services externally in Minikube
- **Step 5: Test in Browser:** Access:

  ```
  http://man-app-demo.local/
  http://man-app-demo.local/api
  ```

  - Verifies path-based routing is working correctly

Key Takeaways

- Ingress = **Layer 7 (HTTP/HTTPS) routing**
- Provides:
  - Single entry point
  - Routing (path + host-based)
  - SSL termination
- Requires an **Ingress Controller** to function
- Routes traffic:
  Ingress → Service → Pods

Storage and Persistence in Kubernetes

Following the Ingress section, the video concludes with the **Storage and Persistence** section (2:48:55 - 3:05:10). Here is a detailed breakdown of the concepts and procedures discussed:

The Core Problem: Data Persistence

- **The Issue:** The instructor explains that by default, files stored inside a container are lost if the container crashes, is deleted, or restarted. This is due to the ephemeral nature of the *writable layer* in containers (2:49:06 - 2:49:45).
- **EmptyDir:** While the previous setup used `emptyDir` as a temporary storage volume, the instructor notes this is not a production-ready solution because the data is tied to the lifecycle of the *Pod* itself.

Key Concepts: Kubernetes Storage

- **PersistentVolume (PV):** A piece of storage in the cluster that has been provisioned by an administrator or dynamically provisioned using Storage Classes.
- **PersistentVolumeClaim (PVC):** A request for storage by a user. It allows a pod to consume abstract storage resources without needing to know the underlying infrastructure details (e.g., whether it's an AWS EBS volume, a local disk, or NFS).

Implementation Procedures

- **Defining the PVC:** The instructor demonstrates creating a `PersistentVolumeClaim` YAML file. This acts as a "ticket" that the *MongoDB* deployment uses to request persistent storage (2:59:36).
- **Mounting the Volume:**
  - The YAML configuration for the *Deployment* must be updated to reference the PVC.
  - The `volumeMounts` section is defined to map the container's data directory (e.g., `/data/db` for *MongoDB*) to the volume claimed by the PVC (2:59:45 - 3:00:20).
- **Verification:** To prove the persistence, the instructor performs a **rollout restart** (`kubectl rollout restart deploy/mongo`) to show that even after the pod is destroyed and recreated, the data remains intact because it is no longer stored strictly within the pod's ephemeral filesystem (3:00:30 - 3:02:45).

Notes: Click Here

DevOps Full Course 2025 - Part 3
Docker Networking, Nginx Reverse Proxy, and Kubernetes (Ingress, Config, Storage)

Author: Sangam Mukherjee

Date: September 29, 2025